

MOTION DETECTION SYSTEM

Professional System Architecture Document

Project Type	Computer Vision / Real-Time Monitoring
Primary Language	Python
Computer Vision	OpenCV
Input	Live webcam video stream
Output	Motion bounding boxes, warning overlay, Windows notification

Purpose: detect significant movement in a webcam feed by comparing consecutive frames, identify the moving region, display a bounding rectangle, and generate a Windows desktop notification.

Document scope: This architecture describes the current classical computer-vision implementation. It is motion detection, not AI-based person/object recognition.

1. System Overview

The Motion Detection System is a real-time computer-vision application built with Python and OpenCV. It continuously reads frames from a webcam, preprocesses them, compares the current frame with the previous frame, and determines whether meaningful movement has occurred.

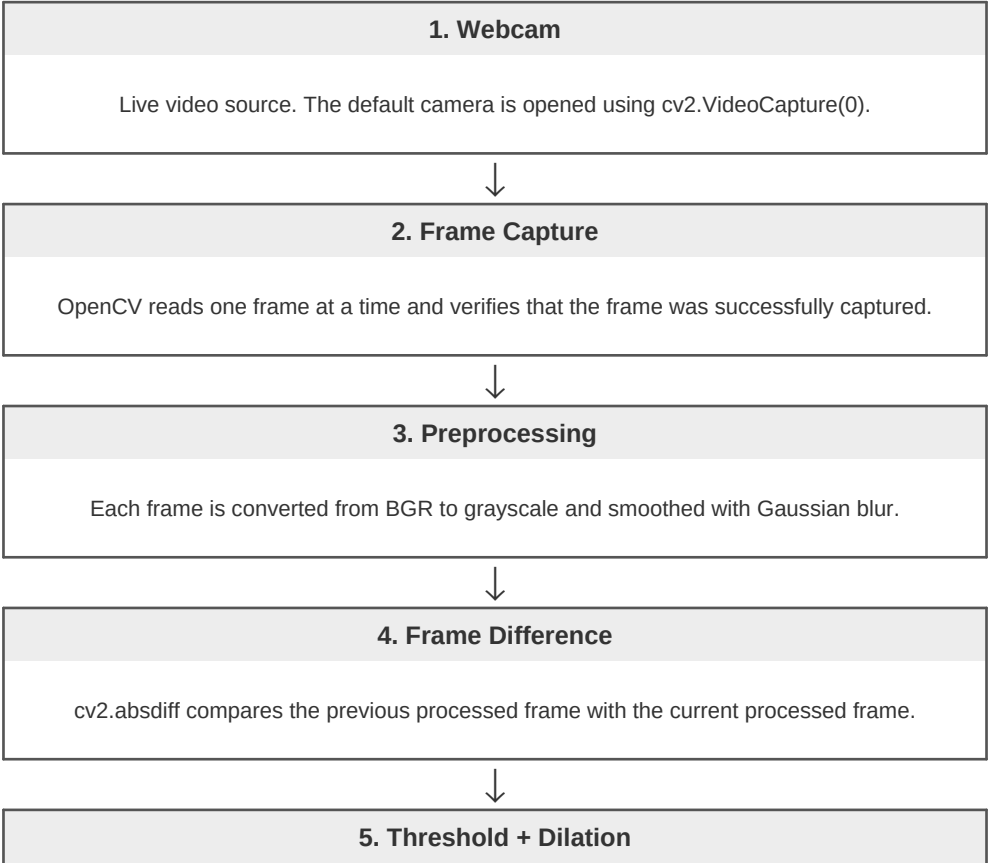
When movement is detected, the application identifies the corresponding contour, calculates its bounding rectangle, draws the rectangle on the live video, displays a warning, and sends a Windows toast notification. When no significant movement is detected, the system displays a normal status message.

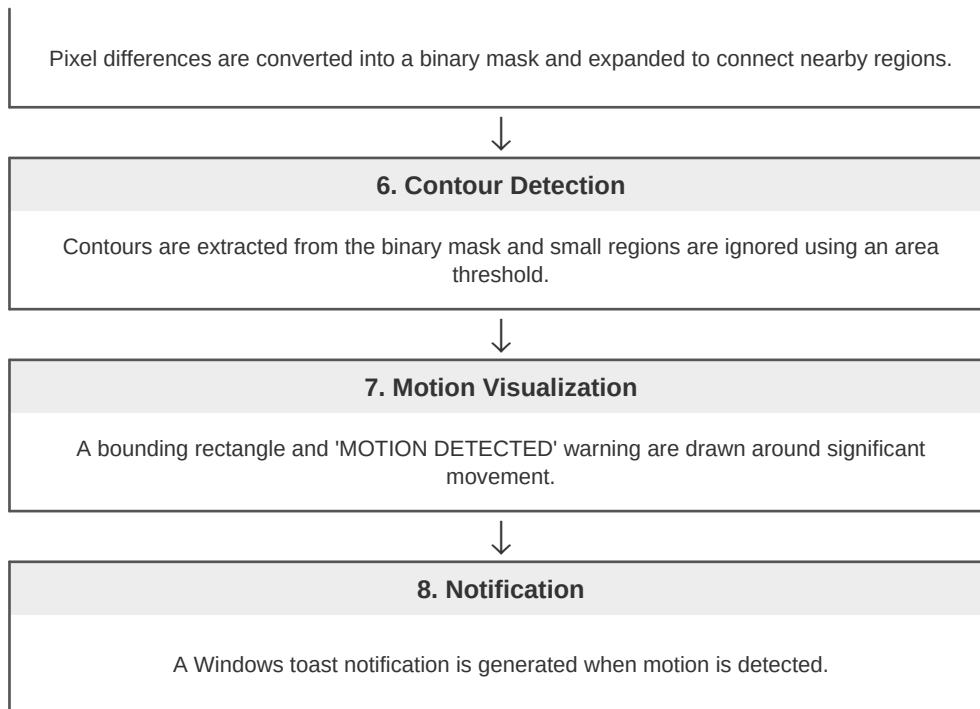
Primary Goals

No.	Goal	Description
01	Capture	Read live video frames from the default webcam.
02	Process	Convert frames to grayscale and reduce noise using Gaussian blur.
03	Detect	Compare consecutive frames and isolate significant changes.
04	Locate	Find contours and calculate bounding boxes around movement.
05	Alert	Show a visual warning and send a Windows notification.

2. System Architecture

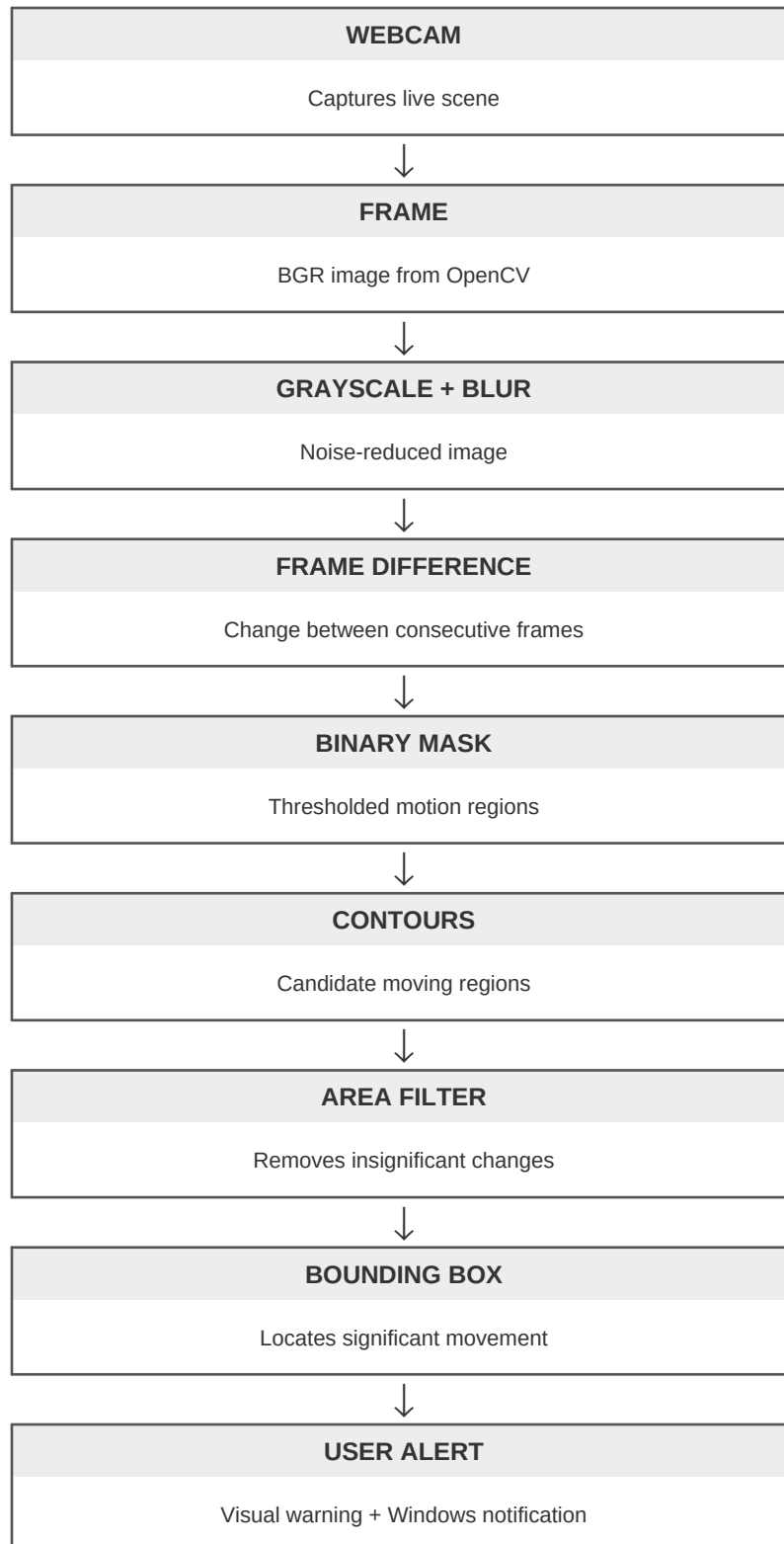
The architecture is organized as a sequential real-time processing pipeline. Each captured frame moves through preprocessing, motion analysis, contour filtering, visualization, and alerting.





3. Data Flow Architecture

The following flow represents how information moves through the system from the physical camera to the user-facing alert.



Real-Time Loop

After processing a frame, the current processed frame becomes the previous frame for the next iteration. This creates a continuous comparison loop until the user presses the configured quit key.

4. Motion Detection Algorithm

The current implementation uses classical frame-difference motion detection. It does not require a trained machine-learning model.

Algorithm Steps

- **Capture:** Read the current webcam frame.
- **Convert:** Convert the frame from BGR to grayscale.
- **Blur:** Apply Gaussian blur to reduce small image noise.
- **Compare:** Calculate the absolute difference between the current and previous frames.
- **Threshold:** Convert the difference image into a binary mask using a threshold value.
- **Dilate:** Expand white regions so nearby motion pixels form connected regions.
- **Find contours:** Detect connected regions representing potential movement.
- **Filter:** Ignore contours below the configured minimum area.
- **Locate:** Calculate x, y, width, and height using a bounding rectangle.
- **Alert:** Draw the rectangle, display warning text, and send a Windows notification.

Core Processing Relationship

Motion = **abs(Current_Grayscale_Blurred – Previous_Grayscale_Blurred)**

Motion Mask = **Threshold(Motion Difference)**

Significant Motion = **Contour Area ≥ Minimum Area**

5. Component Architecture

Component	Technology / API	Responsibility
Camera Input	OpenCV VideoCapture	Opens the webcam and captures frames.
Frame Processing	OpenCV + NumPy	Grayscale conversion, blur, and frame comparison.
Motion Analysis	absdiff + threshold + dilation	Creates a binary representation of significant changes.
Contour Analysis	findContours + contourArea	Finds movement regions and filters small changes.
Visualization	rectangle + putText	Marks movement and displays status on the video.
Notification	Winotify	Sends a Windows desktop toast notification.
Application Control	Python loop + waitKey	Maintains the real-time loop and handles program exit.

6. Technology Stack

Technology	Purpose
Python	Application programming language and control logic.
OpenCV	Webcam capture, image processing, motion detection, contours, and visualization.
NumPy	Underlying numerical array operations used by image-processing operations.
Winotify	Windows desktop toast notification support.
Windows	Target desktop environment for the notification feature.

7. Recommended GitHub Project Structure

A simple repository structure keeps the executable code, documentation, screenshots, and dependency information separated.

```

motion-detection/
├── src/
│   └── motion_detection.py
├── docs/
│   └── Motion_Detection_System_Architecture.pdf
├── screenshots/
│   └── motion_detected.png
├── requirements.txt
├── README.md
└── .gitignore

```

8. Dependencies

The project should list its Python dependencies in requirements.txt so that another developer can reproduce the environment.

```

opencv-python
numpy
winotify

```

Installation example: `python -m pip install opencv-python numpy winotify`

9. Current Limitations

- The system detects visual movement, not specifically people.
- Changes in lighting, shadows, camera movement, or large background changes may trigger motion.
- A stationary person may not be detected after the scene becomes stable.
- The current approach depends on configurable threshold and minimum-contour-area values.
- Windows toast notifications make the current implementation Windows-oriented.

10. Future Improvements

- Add YOLO or another object-detection model to detect people specifically.
- Add notification cooldown/debouncing to prevent repeated alerts.
- Save a snapshot automatically when motion occurs.
- Record a short video clip around each detected event.
- Add configurable motion sensitivity and detection zones.
- Add a web dashboard for remote monitoring.
- Store event timestamps in a database or log file.

11. Architecture Summary

The system follows a lightweight real-time pipeline: **Webcam** → **Frame Capture** → **Preprocessing** → **Frame Difference** → **Threshold/Dilation** → **Contours** → **Area Filtering** → **Bounding Box** → **Visual Alert** → **Windows Notification**. This design is appropriate for a beginner-friendly OpenCV motion-monitoring application and provides a clear foundation for later integration of AI-based person detection.

Architecture principle: Keep image acquisition, processing, detection, visualization, and notification responsibilities logically separated so each part can be improved independently.